Contextual Coherence in Sherlock Holmes Paragraph Continuation via GPT-2 Fine-Tuning

Student One
Student Two
Student Three
Student Four

Abstract

We study next-paragraph generation in Sherlock Holmes stories using a controlled set of GPT-2 fine-tuning experiments. Given the first k paragraphs of a passage, the model has to produce the next one. We keep the backbone fixed at `distilgpt2` and vary prompt format, context length, tuning strategy, and an auxiliary ranking loss. The five main experiments (E1–E5) are evaluated with target-conditioned perplexity, ROUGE-L, BERTScore, and an entity-overlap diagnostic, using three decoding seeds for generation metrics. The pattern is modest but consistent: structured formatting helps a little, four paragraphs of context help more, and E3 is the strongest run overall. LoRA is much weaker than full fine-tuning here, and the ranking objective does not improve on the best baseline. A retrieval variant is included in Appendix A, where it also trails E3.

1 Introduction

Neural generators can sound fluent while losing track of what is happening. In a paragraph-continuation task, that weakness is hard to hide: the next paragraph has to follow the scene, the speaker, and the local plot, rather than merely produce plausible Victorian prose. This makes the task a useful test of coherence beyond sentence-level fluency.

The Sherlock Holmes stories are a convenient testbed because the narrative world is stable. Holmes, Watson, the London setting, and the style recur across the texts. When a continuation drifts away from that world, the error is usually visible rather than subtle.

We ask a deliberately narrow question: which model-side choices matter for paragraph continuation in this setting? The main comparison does not rely on retrieval or prompt engineering tricks. It focuses on context serialization, context length, full fine-tuning versus LoRA, and a simple auxiliary ranking loss.

The mainline follows four pairwise comparisons. E1 versus E2 asks whether structured paragraph markers help more than plain concatenation. E2 versus E3 asks whether a longer context window helps. E3 versus E4 compares full fine-tuning with LoRA under the same long-context structured setup. E3 versus E5 tests the auxiliary ranking objective against the strongest no-auxiliary baseline. We also run E5W as a robustness check for the auxiliary weight, but we do not treat it as a separate mainline result.

The results are not dramatic, but they are fairly easy to read. Structured input gives a small gain. Longer context gives the clearest gain, making E3 the strongest mainline model. LoRA performs worse than full fine-tuning, and the auxiliary ranking loss does not improve on that baseline. Retrieval is kept in the appendix because it does not change the conclusion.

2 Related Work

This project is closest to work on autoregressive transformers for conditional text generation. These models fit continuation tasks naturally: they generate left to right, and pre-trained GPT-style models can be adapted with ordinary fine-tuning [7, 5]. The choice is also practical. A small GPT-2 variant is large enough to show differences between settings, but still small enough for repeated Colab runs.

The project also relates to long-form and story generation. Prior work has shown that local fluency does not guarantee global coherence [1, 2]. Paragraph continuation is a smaller version of the same problem: the output is short, but it still has to respect the preceding discourse.

Parameter-efficient adaptation is another relevant comparison. LoRA updates a small set of low-rank adapters rather than all model weights [3]. Since this project uses limited compute and a small backbone, it is worth checking whether the cheaper tuning method is enough.

Finally, we look at training objectives beyond standard next-token prediction. If the language-model loss is too local to reward paragraph coherence, a ranking signal may help. E5 tests this idea in a simple form by asking the model to prefer the true next paragraph over sampled alternatives. Retrieval is included only as an appendix comparison.

3 Methodology and Architecture

3.1 Task Formulation

We formulate the task as conditional next-paragraph generation. Each example contains the previous k paragraphs and the true next paragraph from the same book. Training still uses token-level supervision, but the behavior we care about is paragraph-level: the continuation should fit the scene, preserve the narrative state, and stay close to the source style.

3.2 Backbone Model

All mainline experiments use `distilgpt2` as the shared backbone. This keeps the comparison controlled and keeps the compute manageable. The model is small enough for repeated Colab runs, but not so small that every setting collapses to the same behavior. By holding the backbone fixed across E1–E5, we can attribute most differences to the design choice being tested.

3.3 Input Representation

The first design choice is prompt format. The plain setting concatenates context paragraphs directly. The structured setting adds explicit paragraph markers. The hope is that these markers make discourse boundaries easier for the model to read, especially when several paragraphs are provided. We test this with E1 versus E2.

3.4 Context Window

The second choice is context length. The short setting uses $k = 2$ paragraphs; the long setting uses $k = 4$. More context should help with scene and character state, but it also leaves less room under the fixed token budget. E2 and E3 isolate this trade-off.

3.5 Fine-Tuning Strategy

The third comparison is full fine-tuning versus LoRA. Full fine-tuning updates all model weights. LoRA trains low-rank adapters while leaving the base model mostly fixed. E3 and E4 test whether that efficiency trade-off works for this small literary dataset.

3.6 Auxiliary Objective

The final comparison changes the objective. The baseline uses the standard causal language-model loss on the target continuation. The auxiliary setting adds a ranking loss, so the model is also trained to score the true next paragraph above sampled negatives. E3 and E5 test whether that extra signal helps.

Table 1: Dataset split summary. Fill the exact train and validation counts before submission.

Split	Samples
Train	5285
Validation	220
Test	663

Table 2: Main experiment matrix.

ID	Goal	Setup
E1	Baseline	$k = 2$, plain, full, no aux
E2	Structure	$k = 2$, structured, full, no aux
E3	Context length	$k = 4$, structured, full, no aux
E4	Fine-tuning	$k = 4$, structured, LoRA, no aux
E5	Objective	$k = 4$, structured, full, ranking

4 Training Methods and Experimental Setup

The implementation uses PyTorch, Hugging Face Transformers, and PEFT. The course permits either TensorFlow or PyTorch; we used PyTorch because TensorFlow GPU support was not reliable across all group members' local machines. To keep the setup consistent, every reported training, generation, and evaluation run was executed in Google Colab.

4.1 Dataset and Preprocessing

We use four public-domain Sherlock Holmes texts from Project Gutenberg: *The Adventures of Sherlock Holmes*, *The Memoirs of Sherlock Holmes*, *The Hound of the Baskervilles*, and *The Return of Sherlock Holmes*. The preprocessing pipeline removes boilerplate material, segments each book into paragraphs, filters out unsuitable paragraphs, and converts the cleaned corpus into supervised $(context, target)$ continuation examples. Each example consists of the previous k paragraphs and the next paragraph from the same book.

The processed dataset contains 5285 training examples, 220 validation examples, and 663 test examples. All reported numerical results use this fixed split.

4.2 Main Experiment Matrix

The main experiment line is designed to isolate one design factor at a time, as shown in Table 2.

An additional run, E5W, repeats E5 with a larger auxiliary weight. This run is not treated as part of the core mainline narrative. Instead, it functions as a robustness check for the auxiliary

setting and supports the final E5 selection using validation-side model comparison.

4.3 Training Setup

Across the mainline experiments, we keep the core hyperparameters fixed: model = `distilgpt2`, epochs = 3, batch size = 2, gradient accumulation steps = 4, learning rate = 5×10^{-5}, warmup steps = 20, maximum sequence length = 512, and training seed = 42. Holding these settings constant helps keep the pairwise comparisons interpretable. Model selection is based on validation performance rather than test-set outcomes. For the E5 versus E5W comparison, we use validation main loss rather than test metrics to decide which auxiliary-weight variant should represent the mainline auxiliary configuration.

4.4 Evaluation Protocol

We report automatic metrics only in the current report version. The headline metrics are target-conditioned perplexity, ROUGE-L [4], and BERTScore F1 [8]. We additionally report entity overlap as a diagnostic metric. It is useful for rough consistency inspection, but it is not treated as the sole basis for major claims because it is a lightweight heuristic rather than a full semantic evaluation.

Generation-based metrics are evaluated with three fixed random seeds (13, 42, and 2026). We report the mean for all metrics and the standard deviation for generation-based metrics. Perplexity is deterministic for a fixed checkpoint and is therefore reported as a mean or single-value quantity across seeds. This evaluation protocol is intended to distinguish stable model differences from small amounts of sampling variance.

4.5 Scope Note

Human evaluation remains deferred. We did run retrieval, but we report it only in Appendix A. The main claims of the paper rest on E1–E5.

5 Numerical Results

5.1 Main Results

Table 3 summarizes the mainline results. Test metrics are reported on the full held-out test set of 663 samples.

For robustness checking, E5W achieves a validation main loss of 3.1062, validation perplexity of 25.8169, test perplexity of 28.5020, ROUGE-L of 0.1111 ± 0.0007, BERTScore F1 of 0.8343 ± 0.0003, and entity overlap of 0.1231 ± 0.0041. Because it does not improve the validation-side selection criterion relative to E5, it is not used as the main auxiliary result in Table 3.

5.2 Pairwise Analysis

E1 vs. E2: input structure. E2 is a little better than E1, but the gap is small. ROUGE-L rises from 0.1113 to 0.1124, BERTScore moves from 0.8341 to 0.8343, and validation main loss drops from 3.1019 to 3.0998. That is enough to say the structured format helps. It is not enough to claim a large change in quality.

E2 vs. E3: context length. E3 is the best model in the mainline. It has the lowest validation main loss (3.0860), the lowest validation perplexity (25.0467), and the lowest test perplexity (27.6525). ROUGE-L and BERTScore stay close to E2, so the gain shows up more clearly in the likelihood-based metrics than in surface-overlap metrics. Even so, the direction is consistent enough to support the longer context setting.

E3 vs. E4: full fine-tuning vs. LoRA. This is the cleanest result in the paper. Under the same long structured setup, LoRA falls well behind full fine-tuning. Validation main loss rises from 3.0860 to 3.2966, test perplexity jumps from 27.6525 to 32.6382, and the overlap metrics drop as well. In this task, with this backbone, full tuning is simply the better choice.

E3 vs. E5: auxiliary ranking objective. E5 does not beat E3. Its validation main loss is higher (3.1022 vs. 3.0860), its validation perplexity is higher (25.6313 vs. 25.0467), and its test perplexity is higher (28.2869 vs. 27.6525). ROUGE-L and BERTScore barely move, while entity overlap drops. The safest reading is that the ranking loss does not help in its current form.

5.3 E5 Robustness Note

We also compare E5 with E5W, which uses a larger auxiliary weight. E5W is slightly worse on validation main loss (3.1062 vs. 3.1022). The difference is small, so E5W still works as a robustness check. It does not change the conclusion.

5.4 Qualitative Examples

Qualitative examples are most useful when they make a numerical difference visible. We would therefore focus on a small set: one example where E3 preserves scene or character state better than E4, one where E3 and E5 are similarly fluent but E5 adds no coherence benefit, and, if space allows, one

Table 3: Mainline results on the held-out test set. Perplexity is deterministic for a fixed checkpoint, while ROUGE-L, BERTScore F1, and entity overlap are reported as mean ± standard deviation over three decoding seeds.

Experiment	Validation Main Loss	Validation PPL	Test PPL	ROUGE-L	BERTScore F1	Entity Overlap
E1	3.1019	25.1105	27.8093	0.1113 ± 0.0009	0.8341 ± 0.0004	0.1435 ± 0.0031
E2	3.0998	25.1061	27.8216	0.1124 ± 0.0002	0.8343 ± 0.0001	0.1434 ± 0.0025
E3	3.0860	25.0467	27.6525	0.1115 ± 0.0003	0.8343 ± 0.0004	0.1266 ± 0.0008
E4	3.2966	30.3848	32.6382	0.1097 ± 0.0005	0.8329 ± 0.0004	0.1275 ± 0.0039
E5	3.1022	25.6313	28.2869	0.1113 ± 0.0012	0.8343 ± 0.0002	0.1207 ± 0.0014

example where structured prompting is visibly better than plain concatenation.

5.5 Limitations

This study has several limits. First, the evaluation is mostly automatic; we do not yet have human judgments of narrative quality. Second, the experiments use one literary domain and one small backbone, so the conclusions should not be read as general claims about story generation. A final caveat is more specific: the best long-context setting still truncates a noticeable share of examples under the 512-token budget. Token-budget inspection gives truncation rates of about 24.7% on the training split and 17.3% on the evaluation split for the $k = 4$ structured setting. This does not overturn the E3 result, but it makes the longer-context gain less clean than it first appears.

6 Conclusion

We ran a controlled set of paragraph-continuation experiments with a fixed `distilgpt2` backbone. The main result is simple: structured formatting helps slightly, longer context helps more, and E3 is the strongest mainline system. LoRA is clearly worse than full fine-tuning in this setting. The auxiliary ranking loss also fails to improve on the best baseline.

For this task and model scale, the most useful change is giving the model a better view of the preceding text. Switching to parameter-efficient adaptation, or adding the simple ranking loss used here, does not pay off.

The next steps are straightforward. Human evaluation would say more about narrative quality than ROUGE or BERTScore can. Larger backbones may also change the trade-offs, especially for LoRA. The appendix retrieval result suggests one narrower lesson as well: retrieval may still help, but not with the simple setup tested here.

Table 4: Appendix retrieval comparison against the strongest mainline model.

Model	Val Loss	Val PPL	Test PPL	ROUGE	BERT
E3	3.0860	25.0467	27.6525	0.1115 ± 0.0003	0.8343 ± 0.0004
Retrieval	3.1759	27.4444	29.8028	0.1089	0.8325

7 Statement of Individual Contributions

Replace the placeholders below with the actual team contribution split before submission.

- Member A: dataset construction, preprocessing, and experiment orchestration.
- Member B: model training pipeline and Colab execution.
- Member C: evaluation pipeline, analysis, and result verification.
- Member D: report writing, literature review, and final integration.

A Retrieval-Augmented Extension

Retrieval is treated as an appendix comparison rather than a main contribution. We ran it separately and compared it with the strongest completed mainline model. It did not win. The appendix system uses a lightweight TF-IDF retriever over earlier passages, following a classical sparse retrieval setup [6].

The retrieval run is weaker than E3 on both likelihood-based and overlap-based metrics. We leave it in the appendix and keep the mainline interpretation unchanged.

B Additional Experimental Details

The appendix can later be expanded with the following material:

- exact dataset statistics and split counts;

- token budget and truncation observations;
- generation settings;
- the detailed E5 versus E5W robustness comparison;
- additional qualitative examples.

References

[1] Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical neural story generation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*, 2018.

[2] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020.

[3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[4] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, 2004.

[5] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. OpenAI Technical Report, 2019.

[6] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5): 513–523, 1988.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[8] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations*, 2020.